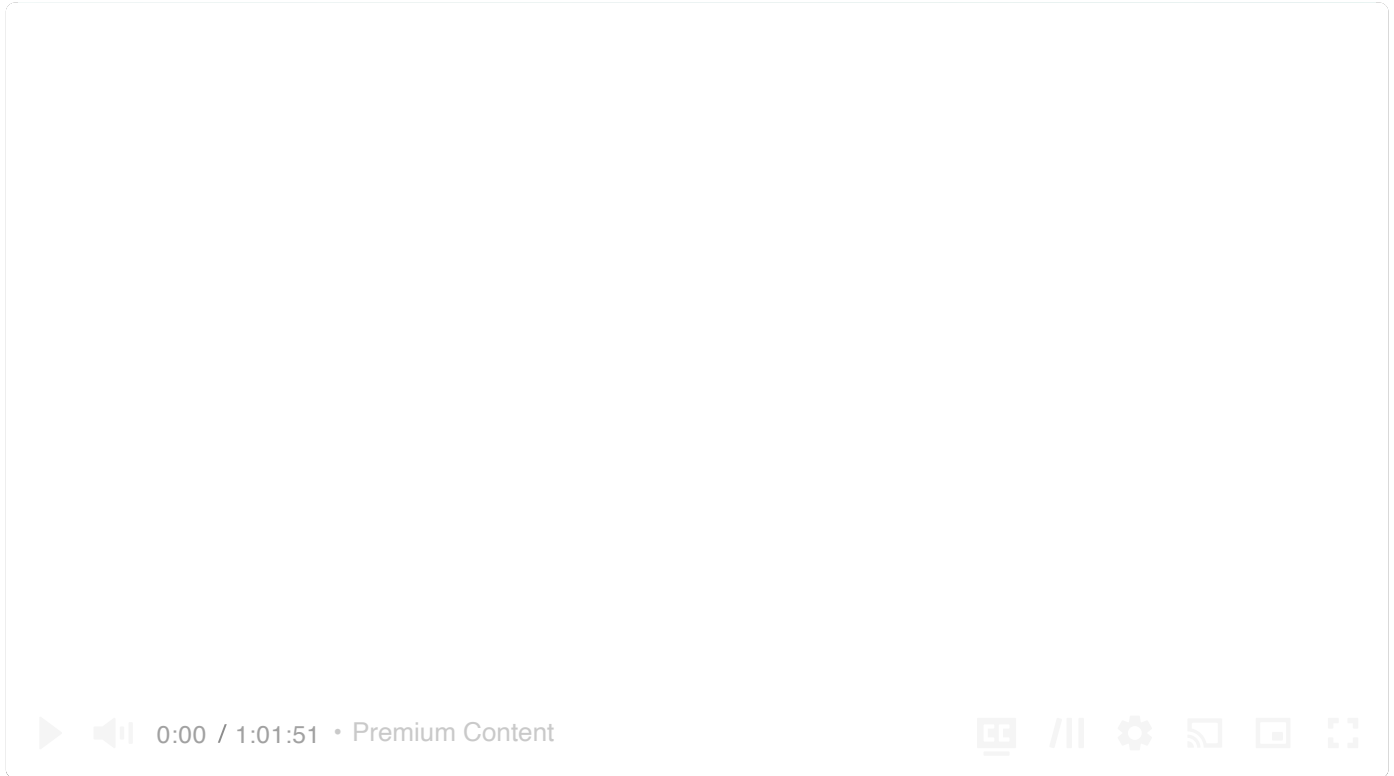




Common Problems

Ask me anything about this topic

Design Google Docs

[⚡ Real-time Updates](#)By Stefan Mai · Published Jul 14, 2024 · 🏷️ **hard** · Asked at: +2

Understanding the Problem

What is Google Docs?

Google Docs is a browser-based collaborative document editor. Users can create rich text documents and collaborate with others in real-time.

In this writeup we'll design a system that supports the core functionality of Google Docs, dipping into websockets and collaborative editing systems. We'll start with the requirements (like a real interview), then move on to complete the design following our [Delivery Framework](#).



Functional Requirements

Core Requirements

1. Users should be able to create new documents.
2. Multiple users should be able to edit the same document concurrently.
3. Users should be able to view each other's changes in real-time.

4. Users should be able to see the cursor position and presence of other users.

Below the line (out of scope)

1. Sophisticated document structure. We'll assume a simple text editor.
2. Permissions and collaboration levels (e.g. who has access to a document).
3. Document history and versioning.

Non-Functional Requirements

Core Requirements

1. Documents should be eventually consistent (i.e. all users should eventually see the same document state).
2. Updates should be low latency (< 100ms).
3. The system should scale to millions of concurrent users across billions of documents.
4. No more than 100 concurrent editors per document.
5. Documents should be durable and available even if the server restarts.



Some non-functional requirements make your job *easier*! In this case, limiting the number of concurrent editors per document is a great constraint to have. It means we can avoid worrying massive throughput on a single document and instead focus on the core functionality.

For what it's worth, this is a choice that Google docs also made - beyond a certain number of concurrent users, everyone new can only join as readers (implying a bit about their architectural choices).

Here's how it might look on your whiteboard:

Functional

- * Users can create new documents
- * Users can edit documents
- * Users can get real-time updates
- * Users can see each others' cursors

Non-Functional

- * Documents are eventually consistent
- * Updates are low latency (100ms)
- * Millions of users, billions of documents
- * No more than 100 concurrent editors
- * Durable documents



Requirements

Set Up

Planning the Approach

Before we start designing the system, we need to think briefly through the approach we'll take.

For this problem, we'll start by designing a system which simply supports our functional requirements without much concern for scale or our non-functional requirements. Then, in our deep-dives, we'll bring back those concerns one by one.

To do this we'll start by enumerating the "nouns" of our system, build out an API, and then start drawing our boxes.

Defining the Core Entities

First we'll define some core entities. These help to define terms between you and the interviewer, understand the data central to your design, and gives us a clue as to our APIs and how the system will fit together. The easiest way to do this is to look over your functional requirements and think about what nouns are involved in satisfying them. Asking yourself deeper questions about the behavior of the system ("what happens when...") will help you uncover the entities that are important to your design.

For this problem, we'll need just a few entities on our whiteboard:

Editor
Document
Edit
Cursor 🔍

Core Entities

We'll explain these to our interviewer as we go through the problem.

- **Editor:** A user editing a document.
- **Document:** A collection of text managed by an editor.
- **Edit:** A change made to the document by an editor.
- **Cursor:** The position of the editor's cursor in the document (and the presence of that user in a document).

Defining the API

Next, we can move to the APIs we need to satisfy which will very closely track our functional requirements. For this problem, we probably want some REST APIs to manage the document itself. We also know we're going to need lots of bi-directional communication between each editor and the document they're collectively editing. In this case it makes sense to assume we'll need some sort of websocket-based approach for the editor experience, so we'll define a few of the messages that we'll send over the wire.

Each interviewer may put more of less emphasis on this step. Some interviewers may want to really understand the intricacies of the messages (veering into a low-level design question) while others may be happy to know that you have a few messages defined opaquely and can move on. We'll assume the latter here!

```
POST /docs
{
  title: string
} -> {
  docId
}

WS /docs/{docId}
SEND {
  type: "insert"
  ....
}

SEND {
  type: "updateCursor"
  position: ...
}
```

```

}

SEND {
  type: "delete"
  ...
}

RECV {
  type: "update"
  ...
}

```



When your API involves websockets, you'll be talking about *messages* you send over the websocket vs endpoints you hit with HTTP. The notation is completely up to you, but having some way to describe the protocol or message schema is helpful to convey what is going over the wire.

High-Level Design

With a sketch of an API we'll move to our high-level design. Tactically, it can be helpful to tell your interviewer that you're sure there are more details of the API to work through but you'll come back to those later as you flesh out the problem.

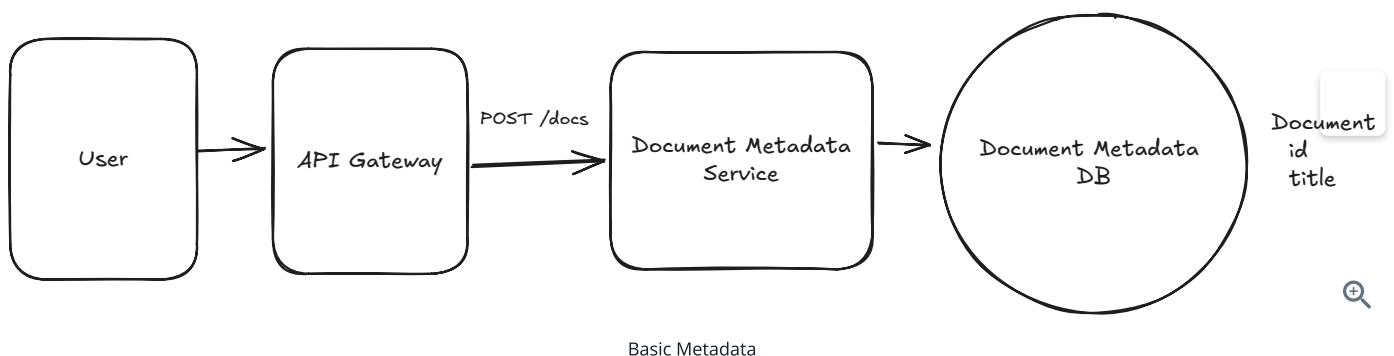


While it's *awesome* to be able to polish a perfect answer, you're in a time crunch! Acknowledging issues that you might get to later and leaving space to adjust your design as you learn more is a great way to keep the interview moving productively.

1) Users should be able to create new documents.

Our first step in our document editor is to allow users to create new documents. This is a simple API that takes a title and returns a document ID. There's a lot of interesting metadata that we may want to hang off of the document (e.g. permissions, history, tags, etc.) so it's sensible for us to assume there will be some separation of the document itself and the metadata. For this API, we'll tackle the first part.

From an interviewing perspective the pattern we're using here of a simple horizontally scaled CRUD service fronted by an API gateway is so common you should be able to sling them out really quickly! Don't spend too much time setting the stage.



Our interviewer is likely to ask us what database we're going to use here. For this problem, let's assume a simple Postgres database for now because it gives us flexible querying and, if we need to scale, we can always partition

and replicate later.

2) Multiple users should be able to edit the same document concurrently.

For our next requirement, we'll have to deal with writing to the document itself which has both consistency and scaling problems. This is where things start to get interesting. In a collaborative editor multiple users are making high frequency edits to the same document at the same time — a recipe for consistency problems and contention!

We're going to defer the scaling concerns for our deep dive later in the interview, so we'll make a note to our interviewer that we'll come back to that so we can focus on consistency.

Collaborative Edits Breakdown

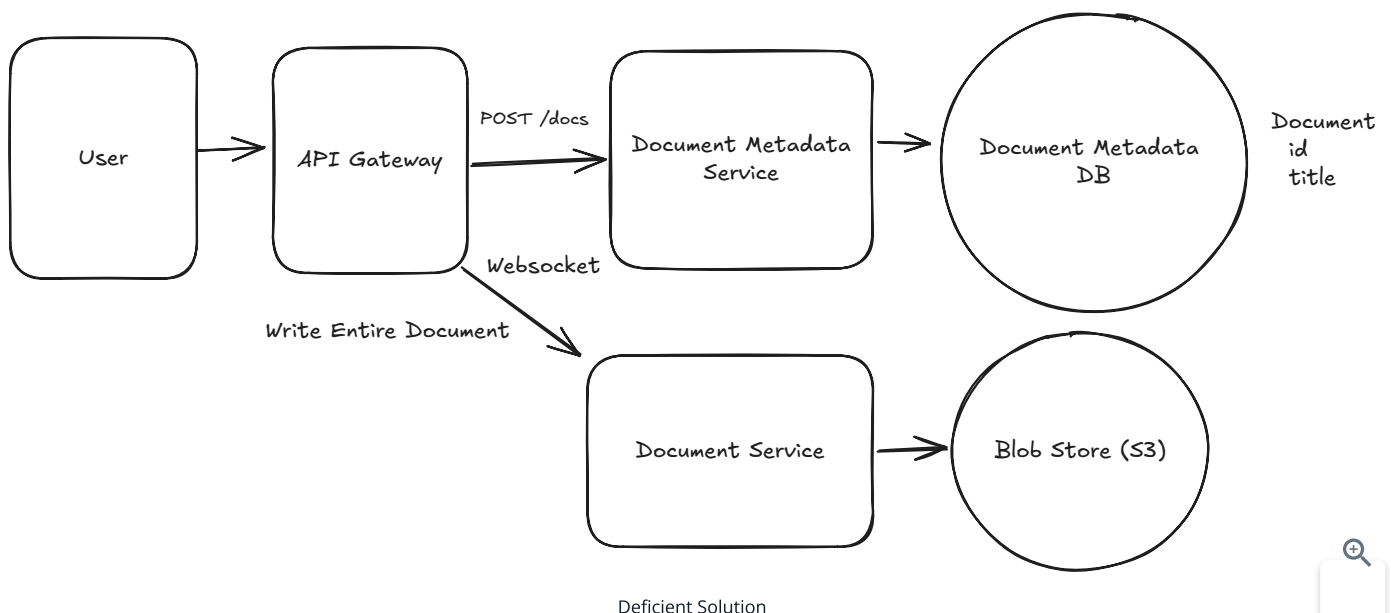
First, let me explain why achieving consistency in a collaborative editor is not easy by starting with a deficient solution and then building up to a more correct one.



I'm going to walk through the fundamentals here to explain how we can solve the consistency problems here but, like before, this is a place where it's good to have the background knowledge to be able to field questions from your interviewer but you probably won't use the interview time to teach these concepts to your interviewer. Skip ahead if you already understand them!

Sending Snapshots (Wrong)

Let's pretend on each edit the users send their entire document over to a `Document Service`. The `Document Service` will then store the document in a blob storage system like S3.



Simple enough, right? Not so fast. First, this design is incredibly inefficient: we're transferring the entire document over the wire for every edit. For a fast typer, this could mean 100s of KBs per keystroke, yuck. But a more serious problem looms.

Assume User A and User B are editing the same document concurrently. The document starts with a single paragraph of text:

Hello!

- User A adds ", world" to produce `Hello, world!` which they submit.
- User B deletes the "!" to produce `Hello` which they submit.

Both submit their edits at the same time. The actual document that gets stored to our blob storage is entirely dependent on which request arrives last. If User A's edit arrives last, the document will be `Hello, world!` If User B's edit arrives last, the document will be `Hello` . The user experience is terrible, we're losing all concurrent edits!

Sending Edits (Getting Warmer)

We can take a crack at solving these problems by recognizing that we're making **edits** to the document. Instead of transmitting the entire document over the wire, we can transmit just the edits themselves.

- User A adds ", world" and sends `INSERT(5, ", world")`
- User B deletes the "!" and sends `DELETE(6)`

We solved our first problem! We're no longer transmitting the entire document over the wire, so now we don't need to send 100s of KB on every keystroke. But we have a new problem.

User B's deletion assumes that the character at position 6 is an exclamation mark. If User A's edit arrives after User B's deletion, we'll end up with `Hello, world` - ok, good. But if User B's edit arrives after User A's edit, we'll end up with `Hello world!` - we deleted the comma instead of the exclamation mark!

Collaborative Editing Approaches

The critical missing piece here is that each edit is contextual: it's an edit based on a specific state of the document. Dealing with a stream of edits presents a bunch of consistency problems! We have two options to solve this problem:

Good Solution: Operational Transformation (OT)



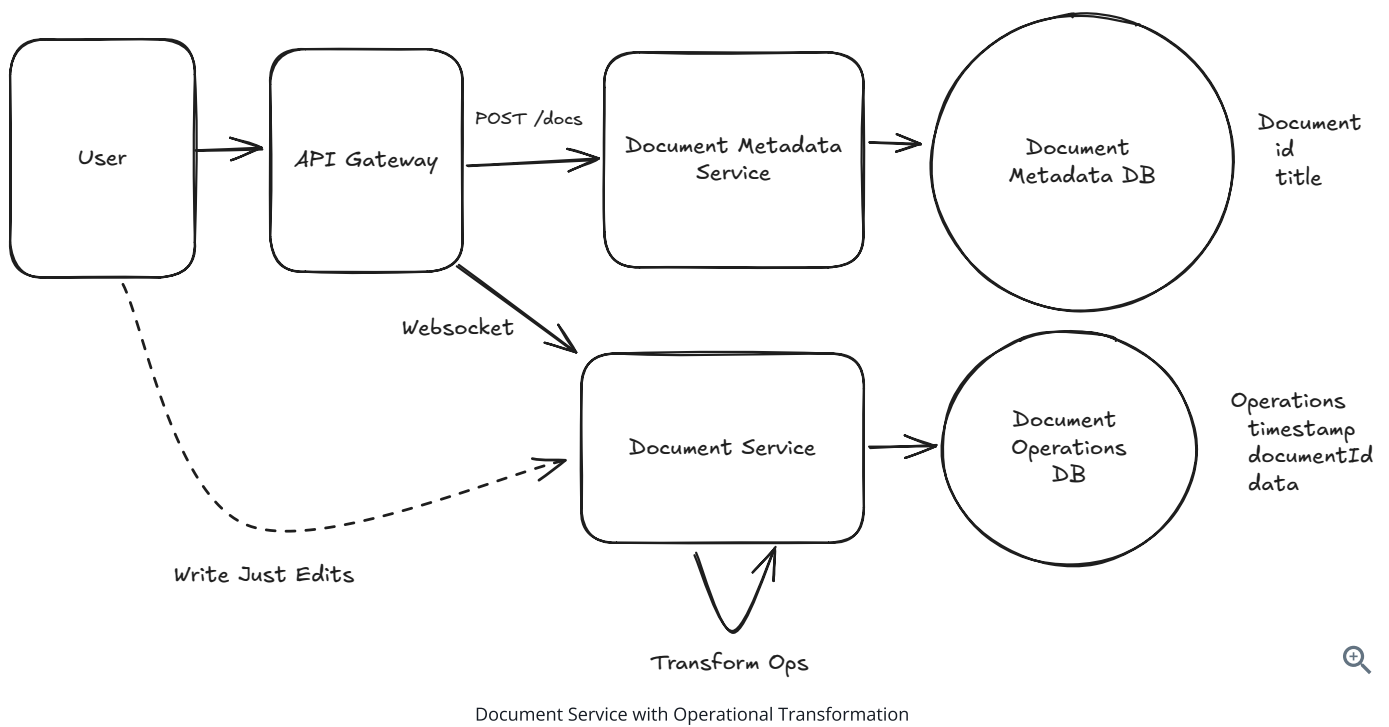
Good Solution: Conflict-free Replicated Data Types (CRDTs)



For this problem, we're going to use the Operational Transformation (OT) approach. This is the approach that Google Docs takes and it benefits from requiring low memory, being more adaptable to text in particular, and making use of a centralized server.

If we needed to support a larger number of collaborators, or a Peer-to-Peer system (maybe using WebRTC!), we could use CRDTs. There are plenty of examples of industrial CRDTs which cut some corners (Figma being a notable one) - there's a healthy amount of research and development happening on both approaches.

With that in mind, we can update our design to solve the collaborative edit problem. Our updated design sends edits to a central server as operations, which are transformed by the server before being recorded in a database.



For our new **Document Operations DB**, we want to use something that we can write very quickly in an append-only fashion (for now). Cassandra should do the trick. We'll partition by `documentId` and order by timestamp (which will be set by our document service). Once the edit has been successfully recorded in the database, we can acknowledge/confirm it to the user. We satisfied our durability requirement!

3) Users should be able to view each other's changes in real-time.

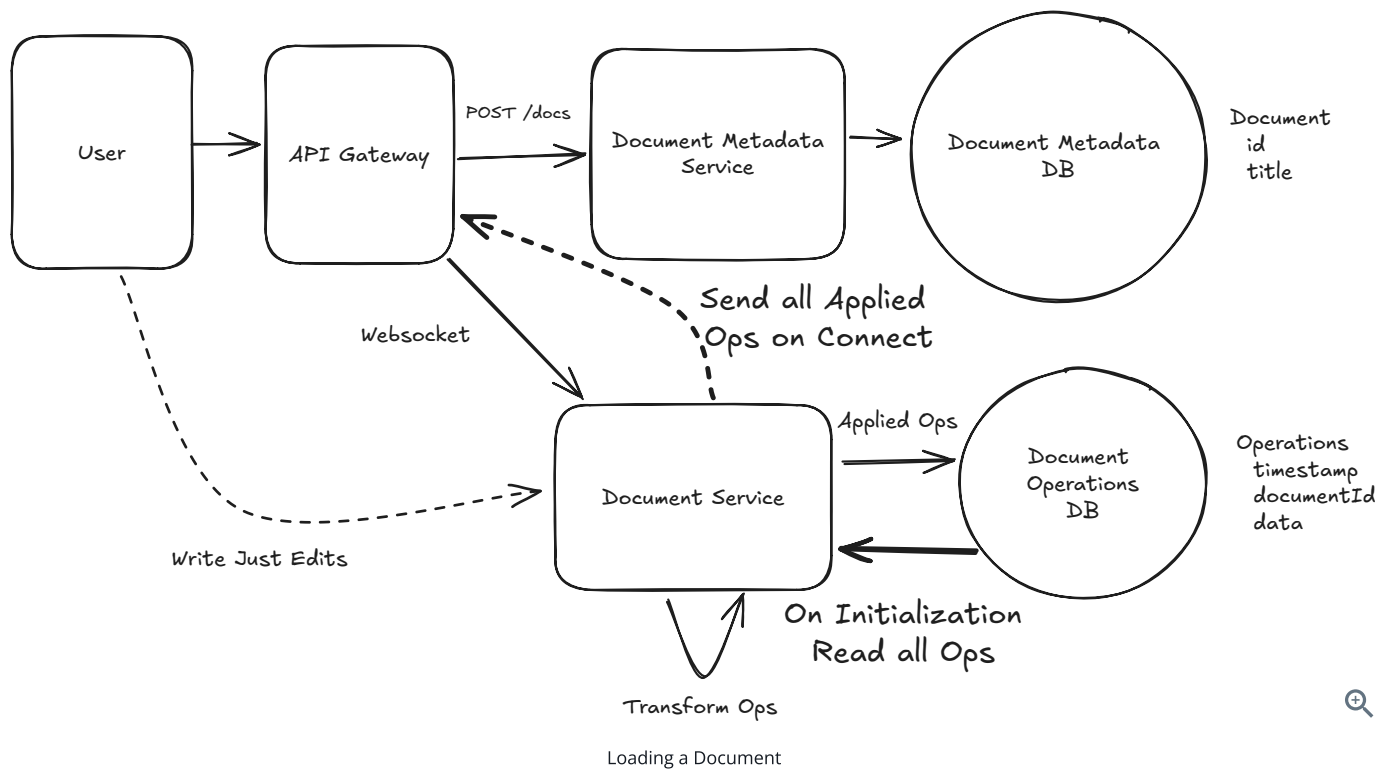
Ok, now that we have a write path let's talk about the most important remaining component: reads! We need to handle two paths:

- First, when a document is just created or hasn't been viewed in a while, we need to be able to load the document from our database and transfer it to the connected editors.
- Next, when another editor makes an edit, we need to get notified so that our document stays up-to-date.

When The Document Is Loaded

When a user first connects, they need to get the latest version of the document. In this case when the Websocket connection is initially established for the document we can push to the connection all of the previous operations that have been applied. Since everyone is connecting to the same server, this allows us to assume that all connections are starting from the same place.





When Updates Happen

When an edit is made successfully by another collaborator, every remaining connected editor needs to receive the updates. Since all of our editors are connected to the same server, this is straightforward: after we record the operation to our database, we can also send it to all clients who are currently connected to the same document.

The next step might surprise you: On the client, we **also** have to perform the operational transformation. Let's talk briefly about why this is.

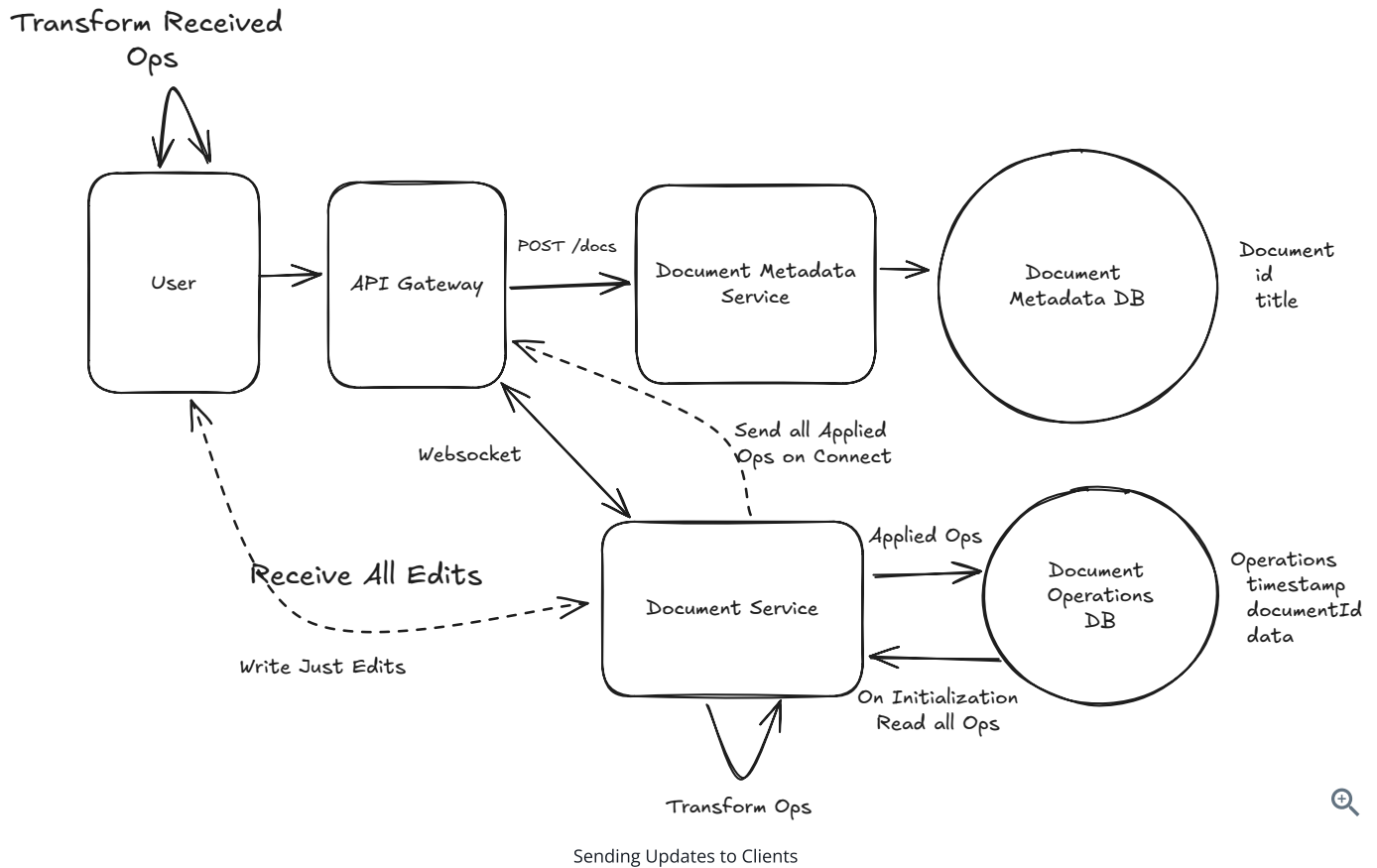
When users make edits to their own document, they expect to see the changes *immediately*. In some sense, their changes are always applied first to their local document and then shipped off to the server. What happens if another user lands an edit on the server *after* we've applied our local change but *before* it can be recorded to the server?

Our OT gives us a way of handling out of order edits. Remember that OT takes a sequence of edits in an arbitrary order and rewrites them so they consistently return the same final document. We can do the same thing here!

So so if User A submits E_a to the server, and User B submits E_b (which arrives after E_a), the perceived order of operations from each site is:

Server: E_a, E_b
 User A: E_a, E_b
 User B: E_b, E_a

Regardless of the ordering, by applying OT to the operations we can guarantee that each user sees the same document!



4) Users should be able to see the cursor position and presence of other users.

One of the more important features of a collaborative editor is "awareness" or being able to see where your collaborators are editing. This helps to avoid situations where you're simultaneously editing the same content (always a mess!) or repeating work. Awareness is inherently transient, we only really care about where our collaborators' cursor is **now** not where it was an hour ago. We also don't care about where the user's cursor is when they aren't currently connected - it doesn't make sense. This data is **ephemeral** with the connection.

These properties help us to decide that we don't need to keep cursor position or the presence of other users in the document data itself. Instead, we can have users report changes to their cursor position to the **Document Service** which can store it in memory and broadcast it to all other users via the same websocket. When a new user connects, the **Document Service** can read the properties of other users out of memory and send them to the connecting user. Finally, the **Document Service** can keep track of socket hangups/disconnects and remove those users from the list (sending a broadcast when it happens to any remaining users).

And with that, we have a functional collaborative editor which scales to a few thousand users!

Potential Deep Dives

With the core functional requirements met, it's time to cover non-functional requirements and issues we introduced.

1) How do we scale to millions of websocket connections?

One of the weakest assumptions we made as we built out the system is that we'll be able to serve everyone from a single **Document Service** server. This won't scale to millions of concurrent users and introduces a single point of

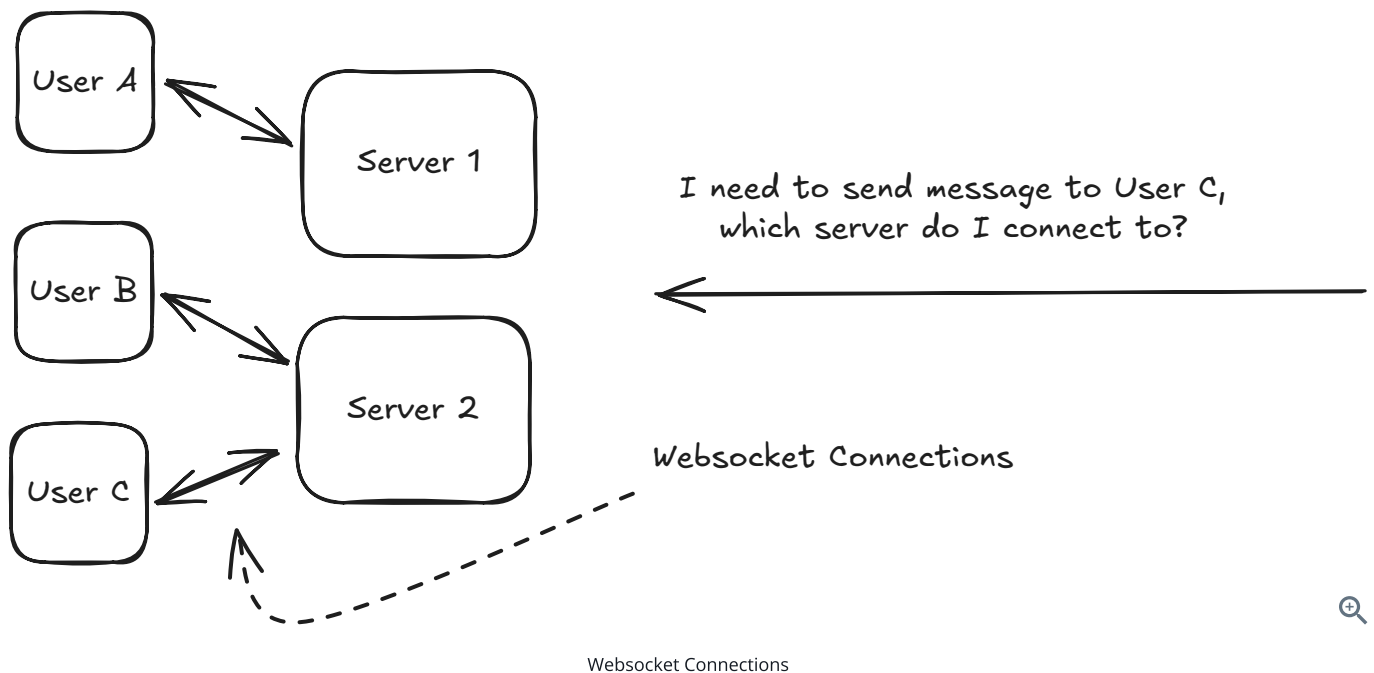
failure for availability.

We need to scale the number of `Document Service` servers to the number of concurrent connections. When I connect to the `Document Service`, I either (a) need to be co-located with all other users connected to the same document or (b) know where all the other users are connected. (See [Whatsapp](#) for a similar problem.)

⚡ Pattern: Real-time Updates

This pattern of needing push updates to clients across a scaled backend service is a quintessential example of the real-time updates pattern.

Learn This Pattern



When you introduce websockets into your design, you're probably doing it because they are *bi-directional*, you can send messages back to your clients. With a traditional client-server architecture you're mostly talking about left-to-right arrows: how can clients connect to the servers to send messages and receive a response. The jump that many candidates fail to make is to think about the **right-to-left** arrows on their diagrams: how do your internal services talk to the clients?

Because the statefulness of websockets is a pain, it can be useful to handle them at the "edge" of your design. By terminating websockets early and exposing an "internal" API to the rest of our system, other systems can retain statelessness and don't need to concern themselves with the details of websocket connections.

The solution here is to horizontally scale the `Document Service` and use a consistent hash ring to distribute connections across servers. Each server in our system joins a the hash ring, with each server responsible for a range of hash values. This means we always know both which server is responsible for the document and all connections go there. We use Apache [ZooKeeper](#) to maintain the hash ring configuration and coordinate between servers.

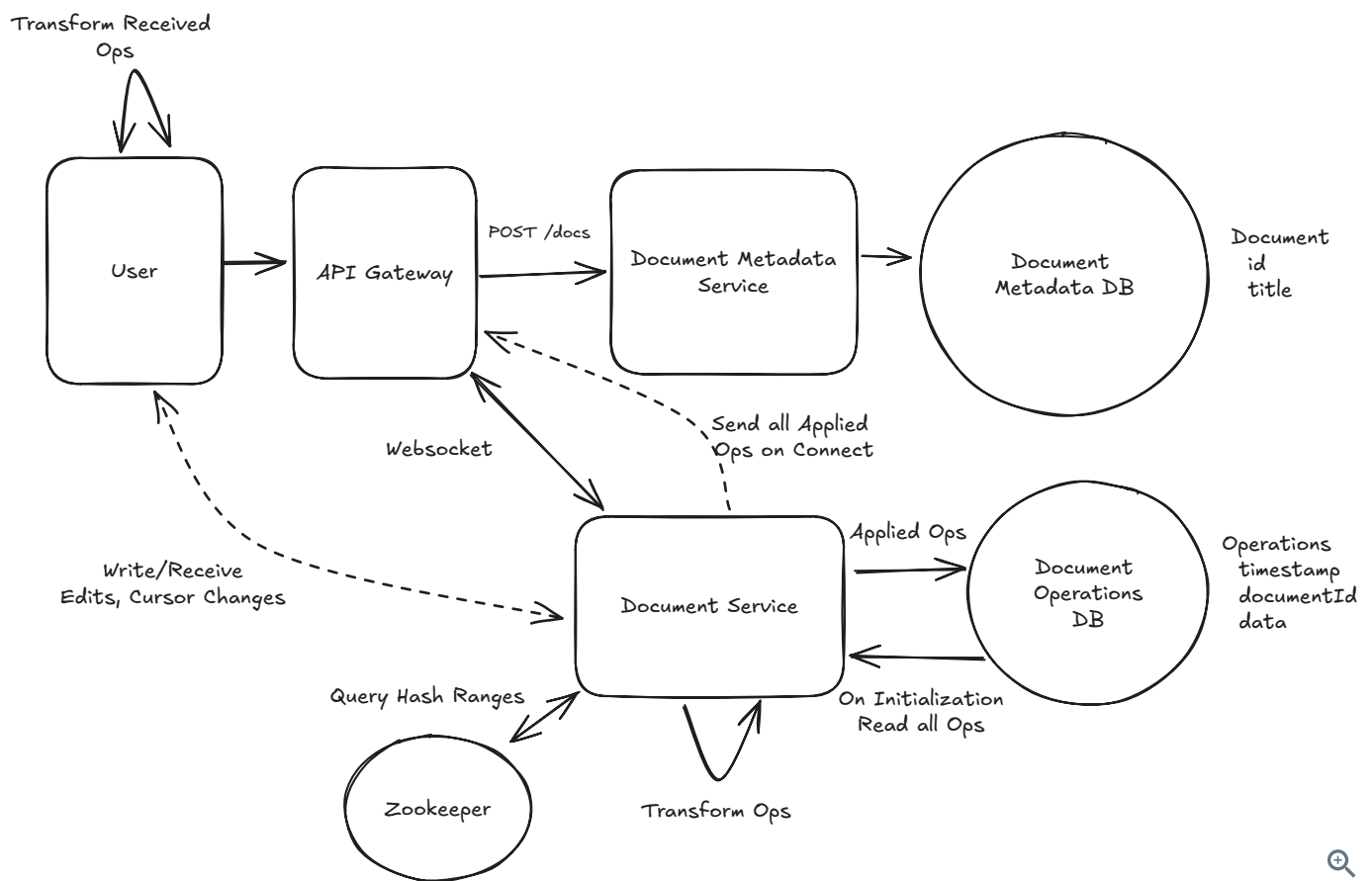
When a client needs to connect:

1. They can initially open an HTTP connection to any of the document servers (potentially via round robin) with the document ID requested.

2. That server will check the hash ring configuration for the given document ID and if it's not responsible for that hash range, it will respond with a redirect to the correct server's address. This way, clients eventually connect directly to the right server without an intermediary.
3. Once the correct server is found, the HTTP connection is upgraded to a websocket connection. The socket server maintains a map from the document ID to the corresponding websocket for use later.
4. We load all the stored ops (if they aren't already loaded) from our **Document Operations DB** and are ready to process operations.

Since all of the users of a document are connecting to the same server, when updates happen we can simply pass them to all connected clients.

The beauty of **consistent hashing** is that when we add or remove servers, only a small portion of connections need to be redistributed. Servers periodically check Zookeeper for ring changes and can initiate connection transfers when needed.



Architecture with Consistent Hash

The downside of this approach is that in these scaling events we need to move a lot of state. Not only do we need to move websocket connections for displaced users (by disconnecting them and forcing them to reconnect to the right server), but we also need to ensure document operations are moved to the correct server.

In an interview, we may be asked to expand on this further:

Scaling complexity

- Need to track both old and new hash rings when adding servers
- Requests may need to be sent to multiple servers during transitions
- Server failures require quick redistribution of connections and hash ring updates

Connection state management challenges

- Need robust monitoring for server failures and connection issues
- Must implement client-side reconnection logic for server unavailability

Capacity planning requirements

- Must ensure servers have sufficient resources for their connection load
- Need to monitor connection distribution to prevent server hotspots

2) How do we keep storage under control?

With billions of documents, we need to be thoughtful about how we manage storage. By choosing OT vs CRDTs, we can already reduce the amount of storage we need by a factor. Still, if each document is 50kb, we're looking at 50TB of storage! Worse, if we have a document with millions of operations, each of these operations will need to be transferred and applied to each client that joins the document!

Remember also that all *active* documents need to be held in memory in our `Document Service`. We need to keep that space small in order to avoid being memory bottlenecked.

One of the most natural solutions here is we'll want to periodically snapshot/compact operations. The idea here is that we don't need all of the operations in our `Document Operations DB` to be stored (exception being if we want to manage some sort of versioning), and we can collapse many operations into one to save on both processing and space.

Lots of options to pull this off!

Good Solution: Offline Snapshot/Compact



Great Solution: Online Snapshot/Compact Operations



Some additional deep dives you might consider

Google Docs is a beast with hundreds to thousands of engineers working on it, so we can't cover everything here. Here are some additional deep dives you might consider:

1. **Read-Only Mode:** Google Docs has a "read-only" mode where users can view documents without interfering with others. It's also considerably more scalable, millions of readers can be viewing the same document. How might we implement this?
2. **Versioning:** Google Docs allows users to revert to previous versions of a document. How can we extend our snapshot/compact approach to support this?
3. **Memory:** The memory usage of our `Document Service` can be a bottleneck. How can we further optimize it?
4. **Offline Mode:** How might we expand our design if we want to allow our clients to operate in an offline mode? What additional considerations do we need to bring into our design?

What is Expected at Each Level?

Mid-level

Personally, I probably wouldn't ask this question of a mid-level candidate. That's not to say you won't get it, but I wouldn't. With that out of the way, I'd be looking for a candidate who can create a high-level design and then think through (together with some assistance) some of the issues that you'll encounter in a high concurrency problem like this. I want to see evidence that you can think on your feet, spitball solutions, and have a small (but growing) toolbox of technologies and approaches that you can apply to this problem.

Senior

For senior candidates, I would expect them to immediately start to grok some of the consistency and durability challenges in this problem. They may start with a very basic, inefficient solution but I expect them to be able to proactively identify bottlenecks and solve them. While I would not expect a candidate to necessarily know the distinction between OT and CRDT, I would expect them to be able to talk about database tradeoffs and limitations, and how they might impact our design. We might have time for an additional deep dive.

Staff

For staff engineers, I'm expecting a degree of mastery over the problem. I'd expect them to be loosely familiar with CRDTs (if they're not, they'll make it up somewhere else) but intimately familiar with scaling socket services, consistency tradeoffs, serialization problems, transactions, and more. I'd expect to get through the deep dives and probably tack on an extra one if we're not too deep in optimizations.

References

- [Real-Time Mouse Pointers](#): How Canva enables collaborative editing using WebRTC.

Mark as read ☐

Add a comment...

B I </> 99 ☰ ☲ 🔗

Comment

☐ Anonymous

Posting as Slim Shady

Q Search comments

Sort By
Popular

 **Rajeev Ranjan**
★ Top 1% • 1 year ago • edited 4 days ago
The difficulty level for this system design problem should be set to hard.
👍 43 [Reply](#)

 **Amir Tugendhaft**
★ Top 5% • 1 year ago • edited 4 days ago

When talking about scaling web-sockets, why not use the same method as in the Live comments answer key:

Redis pub/sub with layer 7 load-balancer to direct you to the relevant server?

And then to topic the pub/sub by the docID.

Are web-sockets and SSE connection different in that aspect?

Here's the link to the live-comments sections:

<https://www.hellointerview.com/learn/system-design/answer-keys/fb-live-comments#1-how-will-the-system-scale-to-support-millions-of-concurrent-viewers>

👍 24 [Reply](#)

I IntellectualCoralWhitefish366

• 1 year ago

+1, seems like the framing in this problem is that we want the ability to broadcast specific events to specific users.

Would it be simpler and more efficient to broadcast the same set of events for each document instead?

👍 0 [Reply](#)



Stefan Mai

Admin • 1 year ago • edited 21 days ago

Yeah good point. The "Great Solution" actually uses docId but the pub/sub variant doesn't. Let me fix this!

👍 2 [Reply](#)



Mohit rao

Premium • 8 months ago • edited 12 days ago

Hey Stefan - not able to see multiple approaches for scaling web-sockets. Is it removed now?

👍 11 [Reply](#)

T

ThoughtfulEmeraldPheasant808

★ Top 5% • 1 year ago

Why can't we use the layer 7 LB for the consistent ring approach as well. That way we have the doc id and the LB can reach out to zookeeper to point to the server that has the respective doc. based on the hash of the doc id. The server can then create a websocket connection with that user and keep the websocket and the userid info in memory.

👍 1 [Reply](#)



Stefan Mai

Admin • 1 year ago

That's what's happening in the great solution? Is the point you're trying to press on the load balancer? Which L7 load balancer did you have in mind?

👍 0 [Reply](#)

T

ThoughtfulEmeraldPheasant808

★ Top 5% • 1 year ago

I was confused with these steps in the great solution

"That server will check the hash ring configuration for the given document ID and if it's not responsible for that hash range, it will respond with a redirect to the correct server's address. This way, clients eventually connect directly to the right server without an intermediary." Why would the server check this. This could be checked by the load balancer by talking to zookeeper before connecting to the server.

👍 0 [Reply](#)

T

ThoughtfulEmeraldPheasant808

★ Top 5% • 1 year ago

I believe Traefik open source L7 and can talk to zookeeper but I am not 100% sure. You are correct that this is fairly uncommon.

👍 0 [Reply](#)**Stefan Mai**

Admin • 1 year ago

It's relatively uncommon to have a load balancer implement logic like this. I can't name one that does. But if you can that's great!

👍 0 [Reply](#)S **stevenjern**

Premium • 8 months ago

When you wrote, "clients eventually connect directly to the right server without an intermediary", did you mean, the client and WebSocket server would connect directly without the API Gateway? Is that secured? also, would each websocket server have its public IPs in addition to private IPs? is that common?

I'm asking because I'm used to architectures where client only connects with LBs created within the demilitarized zone (DMZ), while the backend servers reside in a protected network.

👍 3 [Reply](#)**Rashmi lalwani**

Premium • 7 months ago • edited 28 days ago

The websocket should still be created via APIGateway, L7 load balancer can have a logic on which server to connect based on some server-id which can be passed in the request header. Client can do a http call to the service to get the server id (via Zookeeper) and then make a websocket request with the server-id in the header.

👍 0 [Reply](#)S **SubstantialHarlequinHerring381**

Premium • 1 year ago

Hey Stefan, where is the actual document stored? It's a little unclear from the diagrams, but is it safe to assume that every known version of a document is stored in S3, and when a client connects for the first time, we grab the most recent version from S3 and load the latest operations from Document Ops DB and send this to the client?

👍 10 [Reply](#)**Stefan Mai**

Admin • 1 year ago

In this design, the document is represented as a sequence of operations in the operations DB. After compaction that's just 1 operation "insert {the document contents}".

Having some sort of interweaving with blob storage is a good extension, but remember we're not snapshotting the document here! We're recording a sequence of edits.

👍 13 [Reply](#)O **OkIndigoTiger161**

★ Top 10% • 8 months ago

In Dropbox design, i believe we store the file in S3 due to the file can be very large like 50G. In reality, google doc can also be large. What storage strategy does goold use for large file with collaborative edit

👍 3 [Reply](#)

**Majid Parvin****Premium** • 7 months ago • edited 7 months ago

Hi Stefan,

Imagine we have a document that initially contains the word "hello," and a couple of users go on to make roughly 10,000 edits. The next day, a new team member joins—how can we reconstruct the document's current state if yesterday's updates were never synced to S3?

 1 [Reply](#)**Stefan Mai****Admin** • 7 months ago

The operations are stored in our Document Operations DB (Cassandra), not S3. When loading the document, we'll fetch the latest compacted state plus any subsequent operations - likely far fewer than 10k ops since our compaction service would have collapsed those into a minimal set of operations overnight.

 5 [Reply](#)**deepak gupta****Premium** • 4 months ago • edited 5 days ago

I think this design should be updated with this information. It was unclear to me as well.

I would assume the same. i.e. store the document in S3 and then apply operations on this document when we do compaction.

Whenever some user connects we send the document from S3 and then apply operations with timestamp > timestamp of S3 document. Something like lambda architecture.

 7 [Reply](#)**Amir Tugendhaft**★ **Top 5%** • 1 year ago

Something doesn't sit well with me:

You mention in the CRDT the fact that user A adds , to the text, and it chooses 4.3 randomly as the number between the characters.

But then user B randomly chooses for their edit numbers from 4.1 - 4.8 excluding 4.3 .

If CRDT nodes are supposed to be independent from each other - how come they know not to conflict like that with each other?

And what happens if 4.3 would've been chosen for user B?

 9 [Reply](#)**Stefan Mai****Admin** • 1 year ago • edited 2 days ago

I'm using round numbers for the illustration, but in reality you'd have some random floating point number like 4.3423442346234 which have a significantly lower collision probability. Typical implementations have a tiebreaker scheme based on the site author as well.

 9 [Reply](#)**Amir Tugendhaft**★ **Top 5%** • 1 year ago

Nice. Thanks :)

 0 [Reply](#)**D DecisiveOlivePtarmigan394**

• 1 year ago

I believe each client also uses some jitter so that both are not adding\editing 4.3 , instead they would add jitter so that it is 4.344, 4.44 etc...

👍 3 [Reply](#)

ImprovedMagentaKoala455

Premium • 7 months ago • edited 29 days ago

I don't understand why we need to apply OT at the client side. If there were conflicts can't we just override what we received from the server? Since the server will send OT applied result?

👍 7 [Reply](#)



Pankaj Singh

Premium • 5 days ago • edited 2 days ago

But what if the client has some local changes while the transformed changes from server is in flight and not yet received? You wouldn't want to just overwrite your local changes? So transformation is both needed at client as well as server as far as I believe.

Also, if there is some network connection and you're still writing in your local copy then once the network is back, you might be flooded with huge data from server but still want to retain what you have present in your local copy.

👍 1 [Reply](#)



Ajay Sharma

Premium • 2 days ago

I agree with the point that you raised I worked with the ppt web we have the similar logic on the client which is some sort of the graph and that keeps the revision_ids and based on that we do the merge on the client code as well so that we accidentally don't lose our local changes and get the changes from the concurrent users at the sametime.

👍 0 [Reply](#)

[Show All Comments](#)

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)



Questions

[Meta SWE Interview Questions](#)

[Amazon SWE Interview Questions](#)

[Google SWE Interview Questions](#)

[Engineering Manager \(EM\) Interview Questions](#)

Learn

[Learn System Design](#)

[Learn DSA](#)

[Learn Behavioral](#)

[Learn ML System Design](#)

[Guided Practice](#)

Links

[FAQ](#)

[Pricing](#)

[Gift Mock Interviews](#)

[Gift Premium](#)

[Become a Coach](#)

[Our Coaches](#)

[Hello Interview Premium](#)

Legal

[Terms and Conditions](#)

[Privacy Policy](#)

Contact

[About Us](#)

[Product Support](#)

7511 Greenwood Ave North

Unit #4238 Seattle

WA 98103

© 2025 Optick Labs Inc. All rights reserved.

